Sapienza
Università di Roma

Ingegneria dell'Informazione, Informatica e Statistica

Laurea Triennale

Dipartimento di Informatica

Algoritmi II

Appunti a cura di
Lorenzo Ferraro

10 ottobre 2026

© 2026

Distribuito sotto licenza CC BY-NC-SA 4.0

https://creativecommons.org/licenses/by-nc-sa/4.0/deed.it

Informazioni e Segnalazioni

Questi appunti raccolgono i contenuti del corso di *Algoritmi II*, tenuto nell'ambito del corso di laurea in Informatica presso "La Sapienza".

Il materiale viene mantenuto e aggiornato periodicamente sul mio repository personale `github.com/TonnoArcobaleno/uni-notes`, dove è possibile consultare la versione più recente e proporre correzioni o integrazioni.

Questi appunti non sostituiscono in alcun modo i libri di testo consigliati dai docenti o la partecipazione alle lezioni in aula, ma nascono con l'obiettivo di essere un supporto e agevolare lo studio e ripasso.

Segnalazioni

Trattandosi di appunti scritti e rivisti da uno studente, è possibile che contengano imprecisioni, refusi o passaggi poco chiari. Se ne individui uno, la segnalazione è benvenuta e può avvenire in due modi:

- aprendo una **Issue** sul repository indicato sopra, indicando il capitolo o la sezione interessata e descrivendo brevemente il problema riscontrato;
- scrivendo direttamente a **ferraro.notes.issues@proton.me**, specificando nell'oggetto il documento a cui ci si riferisce e, nel testo, il punto esatto e la natura dell'errore (è il canale preferito, perché più facile che io mi accorga della segnalazione in breve tempo).

Poiché il documento viene revisionato periodicamente, prima di segnalare un problema conviene verificare che non sia già stato corretto nell'ultima versione disponibile sul repository.

Indice

1 Grafi e Algoritmi sui grafi **1**
1.1 Introduzione ai grafi 1
1.2 Problema dei ponti di Konigsberg 2
1.3 Rappresentare i grafi 3
1.4 Cicli 5
1.4.1 Problema sui cicli 6
1.5 Cercare un cammino da x a y 8
1.5.1 DFS 8
1.5.2 DFS nei grafi non connessi 11
1.5.3 DFS Ricorsivo 12
1.5.4 Alberi di ricerca 13
1.5.5 Problema degli esami 14
1.5.6 DFS per i grafi diretti 16
1.5.7 Trovare un ordine topologico in un grafo diretto aciclico 18

Capitolo 1

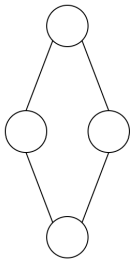
Grafi e Algoritmi sui grafi

1.1 Introduzione ai grafi

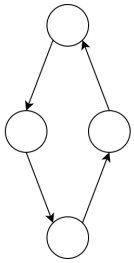
Definizione 1.1 – Grafo

Un **grafo** è una coppia ordinata **G = (V,E)** dove V è l'insieme dei **nodi** (o vertici), mentre E è l'insieme degli **archi** che collegano due vertici.

In questi appunti guarderemo a grafi senza loop (ossia senza archi che collegano un nodo a sè stesso) e senza archi paralleli (ossia grafi che non hanno due o più archi uguali).



Questo è l'esempio più classico di grafo: i nodi sono i pallini, che sono collegati da delle linee che rappresentano gli archi.



Gli archi possono avere anche un orientamento, e se tutti gli archi lo hanno diciamo che il grafo è **orientato**, come il grafo qui a sinistra. In questo caso è possibile avere due archi differenti tra due nodi x e y, ma devono avere la freccia invertita (ossia uno deve portare da x a y e l'altro da y a x).

Proprietà 1.1 – Archi

Preso un arco **f** da x a y, diciamo che

- f è **incidente** a x e y
- x è **adiacente** a y ($x \sim y$)

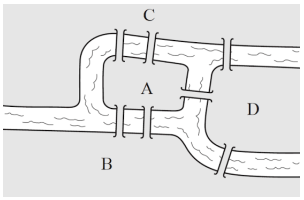
Definizione 1.2 – Grado di un vertice

Il **grado di un vertice** x (**deg(x)**)è il **numero di archi incidenti** ad esso.

Nei due esempi visti prima, il grado di ogni vertice era 2.

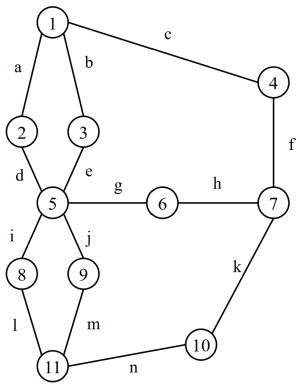
1.2 Problema dei ponti di Konigsberg

Il problema dei ponti di Konigsberg è uno dei problemi logico-matematici più famosi.



La domanda è semplice: con la seguente disposizione di ponti, è possibile attraversarli tutti una volta sola e tornare al punto di partenza?

Questo classico quesito è risolvibile usando i grafi. Infatti, possiamo schematizzarlo in questo modo:

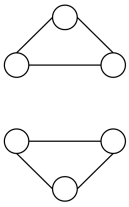


Definizione 1.3 – Passeggiata

Una **passeggiata** in un grafo è una **sequenza** del tipo $v_0 e_1 v_1 e_2 \dots v_{n-1} e_n v_n$, dove v_i e v_{i+1} indicano due vertici collegati dall'arco e_i.
Una passeggiata è detta **chiusa** se $v_0 = v_n$.

Una passeggiata possibile nel grafo qui sopra ad esempio è "1 a 2 d 5 i 8 l 11 n 10 k 7".

Per risolvere il problema dei ponti di Konigsberg bisogna saper rispondere alla domanda "Quand'è che è esiste una passeggiata chiusa in un grafo che contiene tutti gli archi presenti una volta sola?". La risposta è che **qualsiasi grafo con almeno un nodo x tale che deg(x)%2 = 1** (ossia il cui grado è dispari) **non ammette una passeggiata di questo tipo**. Nel nostro caso, quindi, il problema è irrisolvibile: infatti sia il nodo 1, 5, 7 e 11 hanno un grado dispari. Ma se fossero tutti pari? Non è comunque detto.



In questo caso, infatti, nonostante tutti i nodi siano di grado 2 quindi pari, la risposta al problema è comuqnue no. È infatti comuqnue necessario che il grafo sia **connesso**.

Definizione 1.4 – Grafo connesso

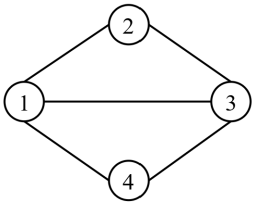
Un grafo si dice **connesso** se per ogni coppia di vertici x e y esiste una passeggiata che porti da x a y.

Proposizione 1.1

Il problema dei ponti di Konigsberg è **risolvibile** $\iff$ **ogni vertice ha grado pari** e il grafo è **connesso**.

1.3 Rappresentare i grafi

Vedremo ora due modi differenti per rappresentare questo grafo:



Definizione 1.5 – Matrice di adiacenze

Si prende una matrice M di dimensione $n \times n$ (dove $n = \#V$) tale che

$$m_{ij} = \begin{cases} \mathbf{1} & \textbf{se } x \sim y \\ \mathbf{0} & \textbf{altrimenti} \end{cases}$$

Nel nostro caso quindi abbiamo una matrice del tipo

$$M = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix} \end{array}$$

Questa matrice, dato che non abbiamo loop nei grafi, avrà sempre la diagonale principale composta solo da 0.

Proposizione 1.2

Se il grafo non è diretto, la matrice di adiacenze sarà **speculare**.

Proprietà 1.2 – Costi computazionali matrice di adiacenze

- **Memorizzazione del grafo**: $\Theta(n^2)$ (spaziale)
- **Visita del grafo**: $\Theta(n^2)$
- **Verificare se $i \sim j$**: $\Theta(1)$
- **Preso un nodo x, calcolare det(x)**: $\Theta(n)$

Definizione 1.6 – Lista di adiacenza

Si prende un'array di dimensione n, e a ogni posizione i dell'array si punta ad una **lista puntata con tutti i vicini del nodo i-esimo**.

Nel nostro caso abbiamo quindi uno schema del tipo

$$\begin{bmatrix} 1 \longrightarrow [2,3,4] \\ 2 \longrightarrow [1,3] \\ 3 \longrightarrow [1,2,4] \\ 4 \longrightarrow [1,3] \end{bmatrix}$$

Proprietà 1.3 – Costi computazionali lista di adiacenze

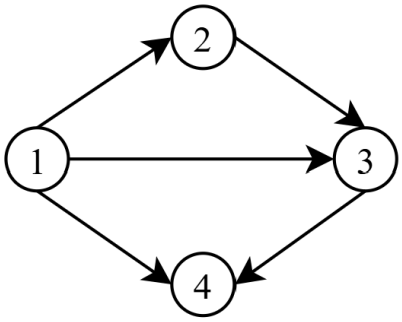
- **Memorizzazione del grafo**: $\Theta(n+m)$ (spaziale), dove $m = \#E$
- **Visita del grafo**: $\Theta(n+m)$
- **Verificare se $i \sim j$**: $O(\min(\deg(i), \deg(j)))$
- **Preso un nodo x, calcolare det(x)**: $\deg(x)$

In realtà, per la memorizzazione e la visita avremmo un costo pari a $\Theta(n + \sum_{v \in V} \deg(v))$, ma dato che $\Theta(n + \sum_{v \in V} \deg(v)) = \Theta(n + 2m) = \Theta(n + m)$, i conti tornano perfettamente.

Per i grafi diretti, invece, ci sono 3 possibilità:

1) Le liste di adiacenze hanno solo i nodi in entrata
2) Le liste di adiacenze hanno solo i nodi in uscita
3) Ci sono due liste di adiacenze, una per i nodi in entrata e una per quelli in uscita

Tutti e 3 i modi sono perfettamente funzionanti. Vediamo con un esempio perché.



Se usiamo le liste di adiacenza con solo i valori in entrata, allora avremo uno schema del tipo

$$\begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix} \begin{matrix} \longrightarrow [] \\ \longrightarrow [1] \\ \longrightarrow [1,2] \\ \longrightarrow [1,3] \end{matrix}$$

mentre quelle con solo valori in output saranno del tipo

$$\begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix} \begin{matrix} \longrightarrow [2,3,4] \\ \longrightarrow [3] \\ \longrightarrow [4] \\ \longrightarrow [] \end{matrix}$$

In entrambi i casi, però, non stiamo perdendo informazioni sul grafo: infatti è possibile da queste liste ricreare il grafo da cui vengono senza alcun problema.

1.4 Cicli

Definizione 1.7 – Cammino

Un **cammino** è una passeggiata che **non ripete i vertici**.

Proposizione 1.3

Presi due nodi x e y, esiste un cammino da x a y $\iff$ esiste una passeggiata da x a y.

Dimostrazione

1° Esiste un cammino da x a y $\implies$ esiste una passeggiata da x a y
È ovvio, dato che un cammino è una passeggiata. ■

2° Esiste un cammino da x a y $\impliedby$ esiste una passeggiata da x a y
Prendiamo la passeggiata di lunghezza minima tra x e y, e ipotizziamo per assurdo che non sia un cammino. Dunque sicuramente esiste un nodo che viene ripetuto, ossia esistono due indici $i < j$ tali che $v_i = v_j$. Possiamo allora tagliare il percorso che porta da v_i di nuovo a sè stesso, dato che stiamo inutilmente allungando il giro. Ma così staremmo accorciando la passeggiata, che abbiamo detto essere di lunghezza minima. Abbiamo quindi trovato un assurdo, dunque abbiamo trovato che la passeggiata più corta è un cammino da x a y. ■

Definizione 1.8 – Ciclo

Un **ciclo** è un grafo connesso dove ogni vertice è di grado 2.

1.4.1 Problema sui cicli

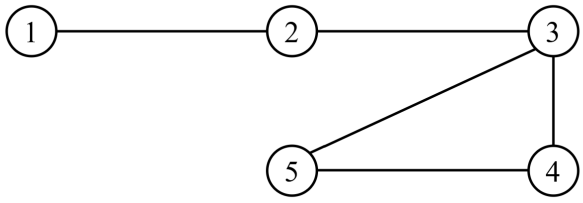
"Dato un grafo dove ogni nodo x ha $\deg(x) \geq 2$, trovare un ciclo". Proviamo a studiare un algoritmo per risolvere questo problema.

Scegliamo un nodo di partenza x. Camminiamo lungo il grafo spondandoci sempre su un vicino diverso dal quale siamo arrivati (il che è possibile dato che $\deg(x) \geq 2$). Prima o poi troveremo un vicino già visitato. in quel momento il nodo su cui ci troveremo avrà due vicini già visitati, il che significa che siamo arrivati ad un ciclo.

```
def trovaCiclo(G):
    L = lista vuota
    x = nodo qualsiasi con grado 2
    L.append(x)
    current = x
    next = x.neighbors[0]
    while(next not in L):
        L.append(next)
        if next.neighbors[0] == current:
            u = next.neighbors[1]
        else:
            u = next.neighbors[0]
        current = next
        next = u
    while L[0] != next:
        L.remove(L[0])
    return L
```

L'ultimo while è fondamentale perché rimuove tutti i nodi che non fanno parte del ciclo.

Vediamo con un esempio grafico perché questo algoritmo funziona, studiando il grafo



Iniziamo con `L = [2]`, `current = 2` e `next = 3`. Dopo il primo ciclo avremo `L = [2,3]`, `current = 3` e `next = 4`. Dopo il secondo ciclo avremo `L = [2,3,4]`, `current = 4` e `next = 5`. Dopo il terzo ciclo avremo `L = [2,3,4,5]`, `current = 5` e `next = 3`. Dato che ora `next` è in `L`, usciamo dal ciclo. Nel secondo while eliminiamo 2, e dunque rimane `L = [3,4,5]`, che è proprio il ciclo nel grafo.

Questa versione, però, non è la più efficiente. Infatti, controllare che un elemento non sia in una lista non ordinata ha un costo $O(n)$, dunque, dato che il primo while è eseguito n volte, arriviamo ad un costo in $O(n^2)$. Vediamo un'altra versione più efficiente.

```
def trovaCicloEfficiente(G):
    n = numero di nodi di G
    L = lista vuota
    Vis = [0] * n
    x = nodo qualsiasi con grado 2
    L.append(x)
    Vis[x] = 1
    current = x
    next = x.neighbors[0]
    while Vis[next] == 0:
        L.append(next)
        Vis[next] = 1
        if next.neighbors[0] == current:
            u = next.neighbors[1]
        else:
            u = next.neighbors[0]
        current = next
        next = u
    while L[0] != next:
        L.remove(L[0])
    return L
```

In questo caso usiamo quindi un array di supporto dove controlliamo se un nodo è già stato visitato o meno, riducendo il controllo della condizione del while da O(n) a O(1), abbassando quindi il costo temporale a **O(n)**. Il costo spaziale invece rimane identico in entrambi i casi: **O(n)**.

1.5 Cercare un cammino da x a y

Presi due nodi x e y in un grafo G, cerchiamo ora un algoritmo per poter trovare un cammino tra loro. Abbiamo due possibilità:

- **DFS** (depth-first-search): partendo dal nodo x, passiamo ad uno dei suoi vicini non visitati fino a trovare un nodo i cui vicini sono tutti già visitati. A questo punto torniamo indietro fino ad un nodo che ha ancora vicini non visitati e continuiamo la ricerca fino a trovare il nodo y.
- **BFS** (breath-first-search): partendo dal nodo x, controlliamo tutti i suoi vicini possibili segnandoli come visitati, e ripeto la stessa cosa per ogni vicino fino a trovare il nodo y.

1.5.1 DFS

Studiamo in particolare l'algoritmo DFS (sulla BFS invece torneremo più avanti).

Abbiamo quindi bisogno di un modo per controllare se un nodo sia stato controllato o meno in tempo costante. Possiamo usare un array `Vis`, di lunghezza n, dove

$$Vis[v_i] = \begin{cases} 1 & \text{se } v_i\text{è stato visitato} \\ 0 & \text{se } v_i\text{non è stato visitato} \end{cases}$$

Per salvare invece il nostro cammino momentaneo usiamo una **pila** (o stack). Questa ci permette di poter aggiungere ad ogni passo il nuovo vertice che stiamo studiando e ci permette di ripercorrere al contrario questo cammino quando siamo arrivati ad un nodo con vicini tutti già visitati. Vediamo quindi un primo algoritmo per la DFS:

```
DFS(G, x):
    n = numero di nodi di G
    Vis = [0] * n
    S = stack nuovo
    S.push(x)
    Vis[x] = 1
    while S != 0:
        y = S.top()
        if esiste z vicino di y tale che Vis[z] == 0:
            S.push(z)
```

```
            Vis[z] = 1
        else:
            S.pop()
    return Vis
```

N.B. Negli pseudocodici che vedremo da qui in avanti utilizziamo la convenzione con indici da 0 a $n-1$. Negli esempi pratici, tuttavia, i nodi dei grafi sono spesso numerati da 1 a n. Per applicare correttamente lo pseudocodice agli esempi sarebbe quindi necessario sottrarre 1 da ciascun nodo, ma per evitare di appesantire i codici noi qui omettiamo questa conversione, lasciando i nodi al loro valore standard.

Proposizione 1.4

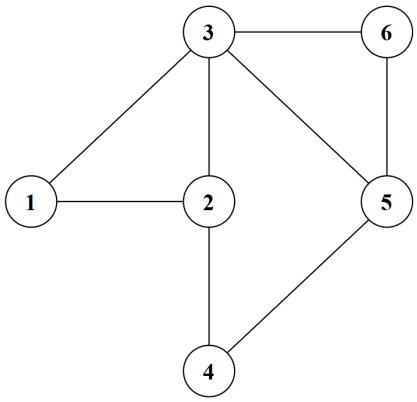
Presi due nodi x e y in un grafo, y è raggiungibile da x $\implies$ **`Vis[y] = 1`**

Dimostrazione

Se y è raggiungibile da x allora sappiamo già che esiste un cammino $xe_1v_1 \dots v_{k-1}e_k y$, ossia un cammino da x a y. Poniamo ora per assurdo che, dopo aver eseguito il nostro algoritmo di DFS, abbiamo `Vis[y] = 0`. Questo significa che esiste un indice i tale che `V[`v_i`] = 1` e `V[`v_{i+1}`] = 0`. Questo significa che v_i è stato visitato, ma dato che l'algoritmo finisce solo quando `S != 0`, significa che v_i è stato tolto dallo stack, ma questo si verifica solo quando tutti i vicini di v_i sono stati visitati. Dunque anche v_{i+1} è stato visitato, e dunque `Vis[`v_{i+1}`] = 1`. Abbiamo quindi trovato un assurdo, dunque nel nostro cammino non esiste un indice tale che `V[`v_i`] = 1` e `V[`v_{i+1}`] = 0`, dunque esiste un cammino tra x e y. ■

Questo algoritmo quindi ritorna un'array `Vis` che ci dice tutti i nodi visitati: non ci dice direttamente se esiste un cammino tra x e y, ma se entrambi i nodi in Vis sono segnati con il valore 1 allora sicuramente questo cammino esiste (possiamo dirlo proprio grazie alla Proposizione 1.4).

Vediamo come funziona il nostro algoritmo con il seguente grafo:



Studiamo come ad ogni iterazione del ciclo while si modificano la pila S e l'array Vis.

1° `S = {1}` e `Vis = [1,0,0,0,0,0]`

2° `S = {1,2}` e `Vis = [1,1,0,0,0,0]`

3° `S = {1,2,3}` e `Vis = [1,1,1,0,0,0]`

4° `S = {1,2,3,5}` e `Vis = [1,1,1,0,1,0]`

5° `S = {1,2,3,5,6}` e `Vis = [1,1,1,0,1,1]`

6° `S = {1,2,3,5}` e `Vis = [1,1,1,0,1,1,1]`

7° `S = {1,2,3,5,4}` e `Vis = [1,1,1,1,1,1]`

8° `S = {1,2,3,5}` e `Vis = [1,1,1,1,1,1]`

9° `S = {1,2,3}` e `Vis = [1,1,1,1,1,1]`

10° `S = {1,2}` e `Vis = [1,1,1,1,1,1]`

11° `S = {1}` e `Vis = [1,1,1,1,1,1]`

12° `S = {}` e `Vis = [1,1,1,1,1,1]` $\implies$ Ritorniamo l'array `Vis = [1,1,1,1,1,1]`

Questo algoritmo ha un costo computazionale pari a $\mathbf{O}(n^2)$. Infatti, il while esegue un totale di $2n-1$ iterazioni, dato che nella pila S vengono inseriti $n-1$ nodi e ne vengono poi eliminati n, e dato che per ogni iterazione abbiamo un costo pari a $O(\deg(y))$ (dato che dobbiamo cercare un vicino z tale che `Vis[z] == 0`, operazione che può richiedere nel caso peggiore $O(\deg n)$), dove y è il nodo in cima alla pila in quella iterazione, che arriva nel caso peggiore a valere $O(n)$, troviamo che $T(n) = O(n^2)$.

Possiamo però migliorare questa complessità temporale tenendo i vicini di ogni nodo in una lista puntata. Vediamo quindi un algoritmo alternativo per la DFS:

```
DFS_Alternativo(G,x):
    n = numero di nodi di G
    Vis = [0] * n
    S = stack nuovo
```

```
    S.push(x)
    Vis[x] = 1
    while S != 0:
        y = S.top()
        while y.adiacenti != 0:
            z = y.adiacenti[0]
            y.adiacenti.remove(z)
            if Vis[z] == 0:
                Vis[z] = 1
                S.push(z)
                break
        if S.top() == y: # y non ha vicini ancora non visitati
            S.pop()
    return Vis
```

A prima vista anche questo algoritmo sembra avere una complessità di $O(n^2)$: infatti, nel while interno iteriamo nel caso peggiore $\deg(y)$ volte, dunque abbiamo per $2n-1$ iterazioni un costo di $O(\deg y) = O(n)$, e quindi $T(n) = O(n^2)$. Ma proviamo a studiarne il costo da un'altra prospettiva: prendiamo un signolo vertice y del grafo e studiamo quanto costa questo vertice all'algoritmo.
Preso y, sappiamo che nel caso peggiore ci costerà al nostro algoritmo $O(\deg(y))$ (ossia quando tutti i suoi vicini sono già stati visitati). Dunque possiamo dire che ogni nodo y ha costo $O(\deg(y))$. Dato che il nostro algoritmo itera 1 volta (in realtà 2, ma sappiamo che una costante moltiplicativa non influisce nel calcolo del costo computazionale, duqnue la ignoriamo) su ogni nodo, dunque possiamo dire che $T(n) = O(\sum_{y \in V} \deg(y)) = O(m)$. Dato che però il nostro while ha costo minimo $O(n)$ (n iterazioni di un while hanno infatti sempre un costo minimo di $O(n)$, che troviamo quando le operazioni all'interno del while hanno costo $O(1)$), possiamo dire che $\boldsymbol{T(n) = O(n+m)}$. Questo ragionamento sul codice visto prima non funziona invece: infatti, per ogni vicino di y li dobbiamo scorrere uno ad uno ogni volta, anche se li abbiamo già visti, portando il costo del singolo nodo a $O(\deg^2(y))$.

N.B. Negli algoritmi futuri useremo quasi sempre come DFS non questo ma l'altro algoritmo in costo $O(n^2)$ per semplicità.

1.5.2 DFS nei grafi non connessi

Definizione 1.9 – Complemento

In un grafo non connesso i suoi **complementi** sono i suoi tanti sottografi connessi.



Ad esempio, in questo grafo vi sono due componenti: uno formato dai nodi 1, 4 e 5, e uno formato dai nodi 2, 3 e 6.

Vogliamo modificare ora la DFS per creare un nuovo algoritmo che ritorni l'array `Comp` lungo n, dove all'indice i vi è segnato il numero del componente in cui si trova il nodo i.

```
DFS_Componenti(G):
    n = numero di nodi di G
    Comp = [0] * n
    x = nodo qualsiasi
    contatore = 0
    while x != -1:
        contatore += 1
        S = stack nuovo
        S.push(x)
        Comp[x] = contatore
        while S!= 0:
            y = S.top()
            if esiste z vicino di y tale che Comp[z] == 0:
                Comp[z] = contatore
                S.push(z)
            else:
                S.pop()
        x = -1
        for i in range(n):
            if Comp[i] == 0:
                x = i
                break
    return Comp
```

Riprendendo il grafo non connesso visto sopra, nella prima iterazione del ciclo while avremmo `Comp = [1,0,0,1,1,0]`, e alla seconda e ultima iterazione avremmo invece `Comp = [1,2,2,1,1,2]`.

1.5.3 DFS Ricorsivo

L'algoritmo DFS può essere riscritto anche come algoritmo ricorsivo:

```
DFS_Ricorsivo(G,x,Vis):
    while esiste un vicino y adiacente a x tale che Vis[y] == 0:
        Vis[y] = 1
        DFS_Ricorsivo(G,y,Vis)
```

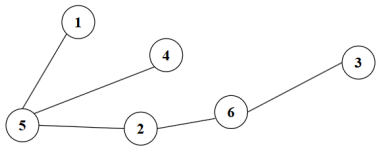
La complessità di questo algoritmo rimane $\mathbf{O}(n+m)$ per lo stesso ragionamento già visto per il `DFS_Alternativo`.

1.5.4 Alberi di ricerca

Definizione 1.10 – Albero di ricerca

Un'**albero di ricerca** è un sottografo aciclico e connesso generato durante una visita (come la DFS) formato solo da archi che portano alla scoperta di nuovi nodi. Ha come **radice** un nodo del grafo e include quindi solo tutti i nodi raggiungibili da tale vertice.

Un albero di ricerca si memorizza tramite **vettore dei padri**, ossia un'array di lunghezza n dove all'i-esima posizione si trova il nodo da cui parte un arco per arrivare al nodo i-esimo.



Ad esempio, in questo albero di ricerca il vettore padri sarebbe `P = [5,5,6,5,5,2]`.

Notiamo come la radice nel vettore padri abbia come valore sé stessa: dunque per trovarla basta solo trovare l'indice i del vettore tale che `P[i] == i`. Notiamo anche che non tutti i valori sono presenti: infatti, mancano l'1 il 4 e il 3. Questo succederà in ogni albero di ricerca: infatti, essendo aciclico, ci sarà sempre almeno un nodo che non avrà alcun "figlio", ossia a cui arriviamo da un suo arco. Questi nodi prendono il nome di **foglie**.

Studiamo ora un algoritmo per trovare la **distanza dalla radice** di un nodo x in un albero di ricerca. La distanza dalla radice è il numero di archi del cammino che porta dal nodo x alla radice.

```
Distanza_Radice(P,x):
    r = 0
    for i in range(len(P)):
        if P[i] == i:
            r = i
            break
    con = 0
    while x != r:
        con += 1
        x = P[x]
    return con
```

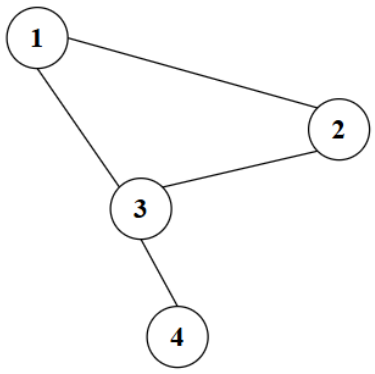
Il costo di questo algoritmo è O(h), dove h indica l'**altezza** dell'albero, ossia la massima distanza tra la radice e una delle foglie. Se ci troviamo in un albero degenere (ossia un albero lineare, dove si ha un numero di archi totali pari a $n - 1$), il costo può arrivare a O(n).

1.5.5 Problema degli esami

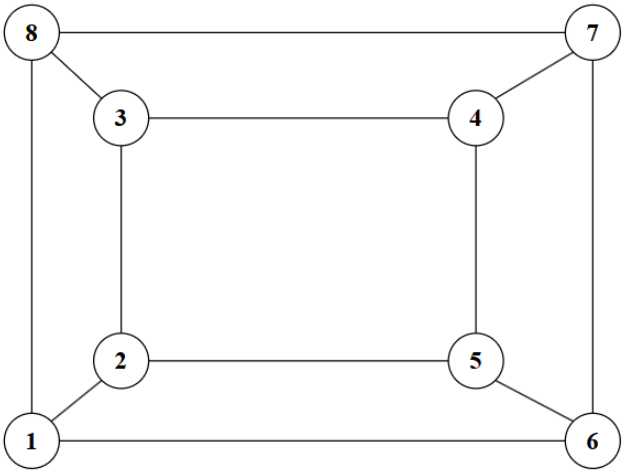
Vogliamo risolvere il seguente problema:

"Presi n esami e m studenti, vogliamo dividere gli esami in due settimane in modo tale che nessuno studente abbia 2 esami la stessa settimana. **N.B.** Uno studente ha sempre due esami da affrontare."

Trasformiamo questo problema in un problema con i grafi: i vertici saranno i diversi esami, mentre gli archi che li collegano saranno gli studenti. Se ad esempio abbiamo 4 studenti e 4 esami, un grafo potrebbe essere il seguente:



Essendoci in questo caso un ciclo a 3, notiamo subito come il problema non sia risolvibile: infatti, i nodi 1 2 e 3 non possono mai stare insieme, ma avendo solo 2 settimane questo non è possibile. Vediamo ora un altro esempio invece:



Partiamo col dire che, preso un nodo x, se esiste una soluzione a questo problema sicuramente esiste una soluzione con x nella prima settimana. Questo è ovvio dato che se esiste una soluzione con x nella seconda, basta scambiare tutti i nodi tra una settimana e l'altra e avremo una soluzione con x nella prima settimana. Dunque possiamo partire col posizionare il nodo 1 nella settimana 1 ad esempio. Ora, tramite una DFS, visitiamo tutti i nodi del grafo e li mettiamo alternati nelle diverse settimane. Troveremo quindi i nodi {1,3,5,7} nella prima settimana e i nodi {2,4,6,8} nella seconda settimana. Questo però non basta: dobbiamo verificare che per ogni arco nel grafo che collega due nodi questi non siano nella stessa settimana. La DFS controlla solo alcuni archi, non tutti, quindi questo controllo è fondamentale. Nel nostro caso la soluzione data non è contraddetta da nessun arco, dunque è corretta. Vediamo come si traduce questo ragionamento in codice.

```
Esami(G):
    n = numero di nodi di G
    x = nodo qualsiasi
    Part = [0] * n
    Part[x] = 1
    S = stack nuovo
    S.push(x)
    while S != 0:
        y = S.top()
        if esiste un vicino z di y tale che Part[z] == 0:
            Part[z] = 3 - Part[y] # In questo modo alterniamo 1 e 2
            S.psuh(z)
        else:
            S.pop()
    for arco uv in G:
        if Part[u] == Part[v]:
```

```
            return false
    return Part
```

1.5.6 DFS per i grafi diretti

L'algoritmo della DFS per i grafi diretti rimane praticamente invariato, se non per la condizione dell'`if`, che diventa `if esiste un vicino con arco (y,z) e Vis[z] == 0`. In questi grafi, però, gli alberi di ricerca prendono il nome di **arborescenza**: è un grafo aciclico connesso che ha per ogni nodo un solo cammino possibile dalla radice a esso.

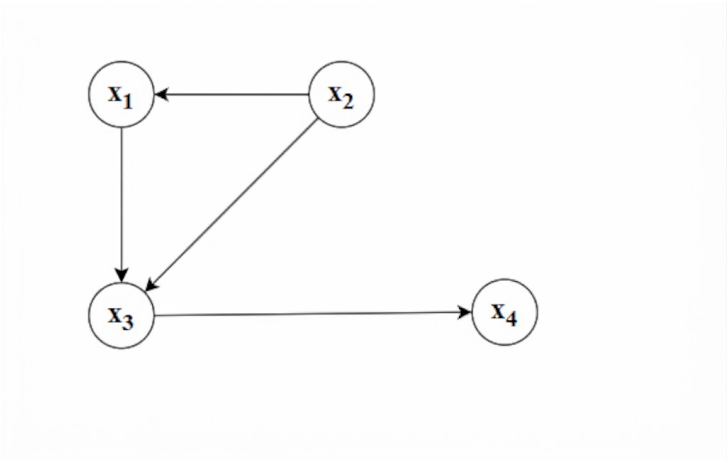
Tramite questi grafi connessi si possono risolvere problemi di questo genere:

"Un processo è diviso in una serie di lavoretti $x_1, \ldots, x_n$ con delle dipendenze tra loro, del tipo x_i va prima di x_j. Trovare un ordine dei lavoretti che rispetti tutte le dipendenze."

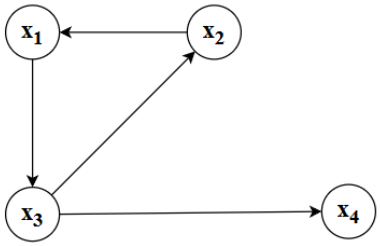
I vertici saranno i diversi lavoretti, mentre gli archi sono le dipendenze. Ad esempio, se x_1 viene prima di x_2, lo indicheremo con un'arco diretto da x_2 a x_1 (indichiamo con la freccia chi viene prima). Se ad esempio abbiamo la lista di lavoretti x_1, x_2, x_3, x_4 con queste dipendenze

- x_1 prima di x_2
- x_3 prima di x_2
- x_3 prima di x_1
- x_4 prima di x_3

allora il grafo sarà di questo tipo:



In questo caso la soluzione sarebbe x_4 — x_3 —- x_1 —- x_2. Questo ordine viene detto **ordine topologico del grafo**. È importante capire che se il nostro grafo avesse avuto un **ciclo diretto** (ossia un cammino dove il vertice iniziale e finale coincidono), allora non avremmo avuto soluzione.



Ad esempio, in questo grafo non è possibile trovare un ordine topologico: i nodi x_1, x_2 e x_3 si trovano in un ciclo diretto, dunque non è possibile stabilire chi debba venire prima.

Proposizione 1.5

Preso un grafo G diretto, se **non ci sono cicli diretti** $\implies$ **esiste un vertice v senza archi uscenti**.

Dimostrazione

Per dimostrare questa proposizione dimostriamo che se ogni vertice ha archi uscenti allora c'è un ciclo diretto. Questo ha senso perché dimostrare che $A \implies B$ è uguale a dimostrare che $\neg A \impliedby \neg B$.

Prendiamo il seguente codice:

```
Trova_Ciclo_Diretto(G): # In G ogni nodo ha un arco uscente
    L = lista vuota
    x = nodo qualsiasi
    L.append(x)
    next = x.neighbors[0]
    while (next not in L):
        L.append(next)
        u = next.neighbors[0]
        next = u
    while L[0] != next:
        L.remove(L[0])
    return L
```

Poiché tutti i nodi hanno un arco uscente, `x.neighbors[0]` è sempre un'operazione sensata. La lista che ci verrà restituita dall'algoritmo è un cammino con il primo e l'ultimo nodo uguali, dunque è un ciclo diretto. Dunque, preso un grafo diretto G con tutti i vertici aventi almeno un arco uscente, questo ha sicuramente un ciclo diretto. ■

1.5.7 Trovare un ordine topologico in un grafo diretto aciclico

Vogliamo ora invece trovare un'algoritmo che trovi un'ordine topologico di un grafo diretto aciclico. Per la Proposizione 1.5 sappiamo che esiste sicuramente un vertice senza archi uscenti. Prendiamo questo vertice, lo segniamo, e lo eliminiamo dal grafo. A questo punto, dato che l'algoritmo non ha cicli diretti, sicuramente vi è un altro nodo che non ha più archi uscenti. Lo prendiamo, lo segniamo e lo cancelliamo, e continuiamo così fino ad aver ricavato un ordine topologico. Se vi sono più nodi senza archi uscenti nello stesso momento basta prendere uno dei nodi in modo casuale: infatti, in uno stesso grafo possono esserci anche più ordini topologici diversi.
Vediamo il codice:

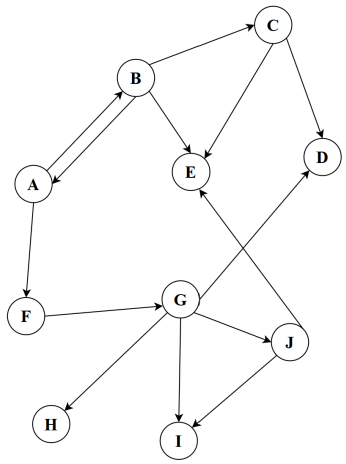
```
Ordine_Topologico(G): # G è un grafo diretto aciclico
    n = numero di nodi di G
    L = [0] * n
    while G != 0:
        v = vertice senza archi uscenti
        L.append(v)
        cancellare v da G
    return L
```

N.B. Questo algoritmo è un algoritmo distruttivo per il grafo: alla fine G sarà vuoto, dunque il grafo va prima salvato per poter eseguire questo algoritmo.

Il costo computazionale di questo algoritmo è $\mathbf{O(n(n+m))}$. Infatti, il ciclo while esegue $O(n)$ iterazioni, e trovare un nodo senza archi uscenti costa $O(n)$, mentre eliminarlo costa $O(m)$ (dato che devo eliminare quel nodo da tutte le liste di adiacenza degli altri vertici). Dunque il costo diventa $O(n(n+m))$.

Il problema è che questo costo è veramente elevato (almeno $O(n^2)$), dunque cerchiamo di studiare un algoritmo che trovi un ordine topologico di un grafo diretto aciclico in tempo $O(n+m)$.

Partiamo sempre dalla DFS, ma aggiungiamo questa volta un contatore che incrementa ogni volta che visitiamo un vertice non ancora visto. Assegniamo poi ad ogni nodo v due valori: $\boldsymbol{t(v)}$ è il valore del **contatore quando visitiamo il nodo v**, mentre $\boldsymbol{T(v)}$ è il valore del **contatore quando eliminiamo quel nodo dallo stack**.



Proviamo ad esempio a calcolare i valori di $t(v)$ e $T(v)$ di alcuni nodi in questo grafico. Ipotizziamo una DFS che segue questo ordine: A,B,C,D,E,F,G,H,I,J. Dunque $t(A) = 1, t(B) = 2, \ldots, t(J) = 10$, ossia ogni nodo v ha come valore $t(v)$ la sua posizione nell'ordine dei nodi visitati. Per quanto riguarda i valori $T(v)$, troviamo ad esempio che $T(D) = 4$: infatti, questo nodo viene inserito e subito eliminato dallo stack dato che non punta a nodi adiacenti non visitati. Abbiamo anche che $T(E) = T(C) = T(B) = 5$: infatti, una volta visitato E questo viene subito rimosso dato che non ha nodi adiacenti non visitati, e vengono poi rimossi anche C e B. Infine, troviamo che $T(J) = T(G) = T(F) = T(A) = 10$: questi sono i nodi che rimangono nello stack una volta che sono stati visitiati tutti i nodi, e che vengono quindi rimossi per ultimi.

Osservazione 1.1

In una **foglia f** abbiamo che $\boldsymbol{t(f) = T(f)}$, mentre nella **radice r** troviamo che $\boldsymbol{t(r) = 1}$ e $\boldsymbol{T(r) = n}$.

Ad ogni nodo v associamo l'intervallo $\boldsymbol{[t(v), T(v)]}$.

Proposizione 1.6

In un grafo diretto aciclico, presi due nodi u e v vi sono tre possibilità:

1) $\boldsymbol{[t(u), T(u)] \subseteq [t(v), T(v)]}$ (**Arco all'indietro**, che va da un discendente a un antenato)
2) $\boldsymbol{[t(v), T(v)] \subseteq [t(u), T(u)]}$ (**Arco in avanti**, che va da un antenato a un discendente)
3) $\boldsymbol{[t(u), T(u)] \cap [t(v), T(v)] = \varnothing}$ (**Arco di attraversamento**, che collega due nodi allo stesso livello, ossia due nodi dove nessuno è antenato dell'altro)

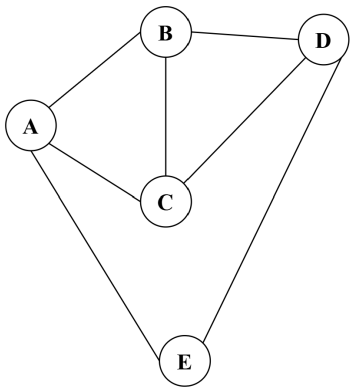
Dimostrazione

Vogliamo quindi dimostrare che è impossibile che gli intervalli di due nodi si sovrappongano, ossia che $t(u) < t(v) \leq T(u) < T(v)$.

Sappiamo che dire $t(u) < t(v)$ equivale a dire che u è stato inserito nello stack prima di v, e dire che $T(u) < T(v)$ significa che u è stato rimosso prima di v dallo stack. Ma queste due proposizioni non possono coesistere: per la natura stessa di uno stack, che è una struttura dati LIFO (Last In First Out), se un elemento entra prima di un altro sicuramente uscirà dopo di

questo. Ciò significa che non esisteranno mai due nodi u e v tali che $t(u) < t(v) \leq T(u) < T(v)$. ■

Seguendo la stessa logica di questa dimostrazione, possiamo dire che preso un **arco** (u, v) ci troviamo sicuramente nella **situazione 2)**. Infatti, sicuramente visitiamo prima il nodo u del nodo v, dunque $t(u) < t(v)$, e dato che ci troviamo in uno stack sicuramente rimuoveremo prima v, dunque $T(v) \leq T(u)$.



È possibile usare questo stesso algoritmo anche nei grafi **non diretti**. In questo grafo ad esempio, seguendo come ordine del DFS A, C, D, B, E, il nodo A ha come intervallo $[1, 5]$, mentre D ha ad esempio $[3, 5]$.

Essendo gli archi non orientati, dei 3 casi visti nella Proposizione 1.6 ne abbiamo ora solo due:

1) $\mathbf{[t(u), T(u)] \subseteq [t(v), T(v)]}$ **(Arco all'indietro)**
2) $\mathbf{[t(u), T(u)] \cap [t(v), T(v)] = \varnothing}$ **(Arco di attraversamento)**

Proposizione 1.7

In un grafo non diretto connesso **non esistono archi di attraversamento al di fuori dell'albero di ricerca**.

Dimostrazione

Poniamo per assurdo che esista un arco (u, v) tale che $[t(u), T(u)] \cap [t(v), T(v)] = \varnothing$. Assumiamo che u venga nell'albero di ricerca prima di v, ossia che $t(u) < t(v)$. Dato che esiste un arco (u, v), non è possibile che il nodo u venga eliminato dallo stack prima della visita del nodo v. Infatti, per eliminare un nodo è necessario aver visitato tutti i suoi vicini, quindi prima di eliminare u dobbiamo per forza di cose visitare v. Dunque sicuramente $t(v) \leq T(u)$, ossia $I_u \cap I_v \neq \varnothing$. ■

Proposizione 1.8

In un grafo connesso non diretto, G ha un ciclo $\iff$ ogni DFS ha un arco all'indietro.

Dimostrazione